

Nmap

A Practical Guide to Network Scanning and Reconnaissance

This guide covers Nmap, the standard tool for host discovery, port scanning, and service enumeration, for use in authorized environments only — personal labs, HackTheBox/TryHackMe boxes, CTFs, and engagements where you have explicit written permission to scan. Scanning networks or systems without authorization is illegal in most jurisdictions.

Contents

- 1. What Nmap Does and Why It Matters
- 2. Installation
- 3. Scan Basics: Targets and Syntax
- 4. Host Discovery
- 5. Port Scan Types
- 6. Service & Version Detection
- 7. The Nmap Scripting Engine (NSE)
- 8. Timing and Performance
- 9. Output Formats
- 10. Firewall & IDS Evasion Options
- 11. Practical Workflow Example
- 12. Good Practice & Legal Notes

1. What Nmap Does and Why It Matters

Nmap (Network Mapper) is a reconnaissance tool used to discover live hosts on a network, identify open ports, determine what services and versions are running on them, and fingerprint the underlying operating system. It's almost always the first step in any penetration test or CTF box — you can't target what you haven't found.

Nmap is generally used to answer four questions about a target:

- Is the host alive and reachable?
- Which ports are open, closed, or filtered?
- What services and versions are listening on those ports?
- What operating system and network configuration is likely in play?

2. Installation

Nmap comes pre-installed on Kali and Parrot OS. On other Linux distributions it's available through the standard package manager:

```
sudo apt install nmap      # Debian/Ubuntu
sudo pacman -S nmap        # Arch/CachyOS
sudo dnf install nmap      # Fedora
```

Some scan types (SYN scans, OS detection) require raw socket access and must be run with sudo.

3. Scan Basics: Targets and Syntax

The general command shape is:

```
nmap [scan type] [options] [target]
```

Targets can be specified several ways:

Target Format	Example
Single IP	<code>nmap 10.10.10.5</code>
Hostname	<code>nmap scanme.nmap.org</code>
CIDR range	<code>nmap 10.10.10.0/24</code>
IP range	<code>nmap 10.10.10.1-50</code>
List from file	<code>nmap -iL targets.txt</code>

4. Host Discovery

Before scanning ports, it often helps to first figure out which hosts on a range are actually up.

Flag	Effect
-sn	Ping scan only — skip port scanning, just report live hosts
-Pn	Skip host discovery entirely, treat all hosts as online (useful when ICMP is blocked)
-PS<ports>	TCP SYN ping on specified ports
-PA<ports>	TCP ACK ping on specified ports
-PU<ports>	UDP ping on specified ports

```
nmap -sn 10.10.10.0/24
```

Many hardened hosts and CTF boxes block ICMP entirely, which makes a plain ping scan report a host as down even when it's up. -Pn is the fix — it's worth trying by default when a target seems unreachable.

5. Port Scan Types

Flag	Scan Type
-sS	TCP SYN scan ("half-open") — fast, stealthy, the default for privileged users
-sT	TCP connect scan — completes the full handshake, used when SYN isn't available
-sU	UDP scan — slower, but necessary for services like DNS and SNMP
-sA	ACK scan — maps firewall rule sets rather than open ports
-sV	Service/version detection layered on top of a port scan
-O	OS detection based on TCP/IP stack fingerprinting
-p-	Scan all 65535 ports instead of the default top 1000
-p 22,80,443	Scan a specific list of ports
-F	Fast scan — top 100 ports only

```
sudo nmap -sS -p- 10.10.10.5
```

6. Service & Version Detection

Knowing a port is open isn't enough — you need to know what's actually running on it before picking an attack path. `-sV` probes open ports and tries to identify the exact service and version.

```
nmap -sV -p 21,22,80,445 10.10.10.5
```

The commonly used combined flag `-sC -sV` pairs version detection with Nmap's default safe scripts, giving a solid baseline enumeration pass in one command:

```
nmap -sC -sV -oN scan.txt 10.10.10.5
```

Version intensity can be tuned with `--version-intensity 0-9` (higher = more probes, slower, more accurate) when the default detection is inconclusive.

7. The Nmap Scripting Engine (NSE)

NSE scripts extend Nmap with automated checks for vulnerabilities, misconfigurations, and deeper enumeration of specific services. Scripts are organized into categories:

Category	Purpose
default	Safe, commonly useful scripts run automatically with <code>-sC</code>
vuln	Checks for known vulnerabilities
auth	Tests authentication mechanisms, default credentials
discovery	Extra host/service discovery information
brute	Brute-force credential testing

```
nmap --script vuln 10.10.10.5  
nmap --script smb-enum-shares -p 445 10.10.10.5
```

Individual scripts can also be found and inspected locally, typically under `/usr/share/nmap/scripts/`.

8. Timing and Performance

Nmap has six timing templates, from paranoid (slowest, stealthiest) to insane (fastest, easiest to detect and most likely to miss results on unstable connections).

Flag	Template
-T0	Paranoid
-T1	Sneaky
-T2	Polite
-T3	Normal (default)
-T4	Aggressive — common choice for labs/CTFs on a responsive network
-T5	Insane

For most home lab or CTF scanning, -T4 is a reasonable default; slower templates matter more when trying to avoid detection on a monitored network.

9. Output Formats

Saving scan results is worth doing by default so you can search and revisit them later.

Flag	Format
-oN file.txt	Normal, human-readable output
-oX file.xml	XML output, useful for parsing or importing into other tools
-oG file.gnmap	Grepable output, easy to pipe into grep/awk
-oA basename	Writes all three formats at once using the same base filename

10. Firewall & IDS Evasion Options

These options exist to test how a target's defenses handle unusual traffic; they're primarily relevant to authorized red-team engagements assessing detection capability, not routine enumeration.

Flag	Effect
-f	Fragments packets to make signature matching harder
-D decoy1, decoy2, ME	Scans alongside decoy IPs to obscure the real source

<code>--source-port <port></code>	Spoofs the source port, sometimes bypassing naive filters
<code>--data-length <n></code>	Appends random data to packets to alter their signature

11. Practical Workflow Example

A realistic sequence for enumerating a single authorized lab target from scratch:

- **Quick pass:** `nmap -Pn -F 10.10.10.5` to get a fast read on the most common ports
- **Full port sweep:** `sudo nmap -Pn -p- -T4 -oN allports.txt 10.10.10.5` to make sure nothing on an unusual port is missed
- **Targeted enumeration:** `nmap -Pn -sC -sV -p <discovered ports> -oN detailed.txt 10.10.10.5`
- **Deeper checks:** run relevant NSE script categories (vuln, service-specific scripts) against any interesting open ports
- **Review:** read through the saved output files for versions and banners that map to known vulnerabilities before moving to exploitation

12. Good Practice & Legal Notes

- Only scan systems you own or have explicit written authorization to test (a signed scope-of-work, a lab like HTB/TryHackMe, or your own VMs/network).
- Unauthorized network scanning can be treated as a computer-crime offense in many jurisdictions, even without exploitation following it.
- Run a full port sweep (-p-) at least once per target — services on non-standard ports are a common miss in CTFs and real engagements alike.
- Save scan output (-oA) by default so you can grep back through results instead of re-scanning.
- Be mindful of scan intensity and timing on production or shared networks; aggressive scans can disrupt sensitive services.

This guide is meant as a working reference alongside hands-on labs (HTB, TryHackMe, personal home networks) and doesn't replace understanding the protocols and services being scanned.